Creditly Technologies

Full Stack Developer

Technical Assignment

Creditly Internal Platform + Banking Auction Module

Overview

- Build an internal system to manage:
  - Users
  - Accounts (customer cases)
  - Events
  - Auction Opportunities
  - Bank Offers
- Focus areas:
  - Backend architecture
  - Role-Based Access Control (RBAC)
  - Business logic
  - Event-driven thinking
  - Integration layer

Objective

Build a system that simulates a real production flow:

- Authentication (JWT)
- Role-based permissions
- Auction system for banks
- Event creation and tracking
- External integration (mock CRM)
- Basic frontend

Tech Requirements

Backend

- Node.js with TypeScript
- REST API
- Clean architecture:
  - Controllers
  - Services
  - Repositories
  - Integration layer

Frontend

- Next.js (App Router)

- TypeScript
- Basic UI only

Database

- SQL or MongoDB or other(your choice, explain in README)

Bonus (optional)

- Queue / retry mechanism
- Cloudflare stack
- Audit trail

Entities

- User – system user
- Account – customer case
- Event – system action
- AuctionOpportunity – auction entity
- BankOffer – bank's offer

Roles

- **Admin**
  - Full access
- **Manager**
  - Access only to assigned accounts
  - Can open/close auctions
- **User**
  - Can create events
  - Access only related data
- **Banker**
  - Can view auction opportunities
  - Can submit offers
  - Cannot access personal customer data

RBAC Rules (must be enforced in backend)

Banker can:

- View only auctions that:
  - Are open
  - Match their bank eligibility
- Submit offers only if:
  - Auction is open
  - Not expired

Banker cannot:

- View:
  - Customer name

    - Phone / email
    - Any identifying data
- Access Accounts directly
- View other banks' offers (depending on chosen model)

Accounts

- List accounts
- View account details
- Show manager and status
- Show auction status (if exists)

Events

- Create and view events
- Required types:
    - document_uploaded
    - status_changed
    - note_added
    - auction_opened
    - offer_submitted
    - auction_closed

Auction Module (Critical)

Flow:

- Account becomes eligible
- Manager opens auction
- Banker sees opportunity
- Banker submits offer
- Auction closes
- Winning offer is selected

Rules:

- Auction duration: 3 days
- No offers allowed after expiration
- Best offer = lowest total interest rate

Auction Models (choose one and explain)

- Blind – no visibility of other offers
- Open – partial visibility of competition
- Sealed – one-time submission

Business Logic (mandatory)

- More than 3 events in 24h → mark account as High Activity
- document_uploaded → update lastActivity + trigger sync
- Auction closed → select best offer (lowest interest rate)

Optional rules:

- Tie → earliest offer wins
- No offers → mark as Expired
- Winning offer → Account marked as Won

Integration Layer

- Dedicated service (not from controller)
- Mock CRM system

Triggers:

- status_changed
- document_uploaded
- auction_opened
- winning_offer_selected

On failure:

- Save failureReason
- Mark syncStatus = Failed

API (examples)

- POST /auth/login
- GET /accounts
- GET /accounts/:id
- POST /events
- POST /accounts/:id/auctions
- POST /auctions/:id/offers
- POST /auctions/:id/close
- GET /analytics/summary

Requirements:

- Validation
- Proper RBAC enforcement
- Error handling

Frontend

- Login page
- Accounts list
- Account details
- Events view
- Auctions view
- Offer submission

Focus:

- Proper API usage
- Role-based UI
- Loading + error states

Testing

At least 5 tests covering:

- RBAC behavior
- Banker cannot see personal data
- Best offer selection
- No offers after expiration
- Integration failure handling

Deliverables

- Git repository
- README including:
  - Architecture
  - Database design
  - RBAC explanation
  - Auction model
  - Assumptions and trade-offs
- .env.example
- UI screenshots

Time Expectation

- 8–10 hours
- Not expected to be perfect - focus on structure and logic

Evaluation Criteria

- Backend architecture – 25%
- RBAC – 25%
- Business logic – 15%
- Auction module – 15%
- Integration – 10%
- Frontend – 5%
- README clarity – 5%

What We Are Looking For

- Strong backend thinking
- Clean architecture
- Proper separation of concerns
- Real RBAC implementation
- Handling of sensitive data
- Event-driven logic
- Ability to make and explain decisions

Important Notes

- UI does not need to be polished
- Code clarity matters
- Explanations matter
- Assumptions are encouraged